

IRONROOT Rust Deterministic Architecture

A Rust-oriented architecture for identity, erasure, cycles, dissonance, and Shadow systems.

Purpose: map the design into deterministic Rust systems: no hidden runtime randomness, no simulation-critical floats, and all run history recorded as event data.

1. Crate Layout

```
ironroot_engine/  
core/ tick, fixed math, ids, rng  
world/ voxel, camera, plane, zone, spirit  
combat/ aspect, stats, resonance, alchemy, collision, damage  
systems/ overcoming, quincunx, yod, erasure, faction  
shadow/ recorder, file, replay, classifier  
progression/ zodiac, endings, names, refusal  
roguelike/ root_cycle, event_ledger, schedules  
save/ checksum, serialization, validation
```

2. Deterministic Tick Kernel

```
pub const TICK_HZ: u32 = 120;  
  
#[derive(Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]  
pub struct Tick(pub u64);  
  
pub fn step_world(world: &mut WorldState, input: FrameCommands) {  
    apply_commands(world, input);  
    resolve_erasure_events(world);  
    resolve_faction_actions(world);  
    resolve_aspects(world);  
    resolve_ai(world);  
    resolve_harmonics(world);  
    resolve_collisions(world);  
    resolve_damage(world);  
    record_shadow_frame(world);  
    world.tick.0 += 1;  
}
```

3. Core Entity Model

```
pub struct IronrootEntity {  
    pub id: EntityId,  
    pub stats: Stats,  
    pub zodiac: Zodiac,  
    pub aspect: CombatAspect,  
    pub body: HarmonicBody,  
    pub identity: IdentitySignature,  
    pub position: Vec3i,  
    pub velocity: Vec3i,  
    pub faction: Option<Faction>,  
    pub ending_pressure: EndingPressure,  
}
```

4. Identity Graph

Identity should be relational. A person is not just an entity ID. A person is a body, name, grave, witnesses, debts, memories, and Shadow relation. This supports legal erasure as a real mechanic.

```
pub struct IdentitySignature {
    pub body_hash: u64,
    pub name_hash: Option<u64>,
    pub grave_hash: Option<u64>,
    pub witness_hashes: Vec<u64>,
    pub debt_edges: Vec<DebtEdge>,
    pub memory_edges: Vec<MemoryEdge>,
    pub shadow_edge: Option<EntityId>,
}

pub enum IdentityIntegrity {
    Whole, Damaged, Erased, Counterfeit, Contextual, Vowless,
}
```

5. Erasure Events

```
pub struct ErasureEvent {
    pub erased_entity: EntityId,
    pub erased_name_hash: u64,
    pub faction_source: Faction,
    pub world_position: Vec3i,
    pub severity: ErasureSeverity,
    pub recoverable: bool,
    pub entropy_cost: u32,
    pub spirit_trace: Option<SpiritTraceId>,
}

pub enum ErasureSeverity {
    DebtMarked, NameDamaged, NameRemoved, SpiritSevered, HistoryCollapsed,
}
```

6. Major Erasure Schedule

```
pub struct MajorErasureEvent {
    pub event_id: u64,
    pub faction: Faction,
    pub target_zone: ZoneId,
    pub target_names: Vec<NameId>,
    pub prime_tick: u64,
    pub warning_tick: u64,
    pub active_tick: u64,
    pub fail_tick: u64,
    pub prevention_flags: Vec<PreventionFlag>,
    pub aftermath_state: AftermathState,
}
```

7. Root-Cycle

```
pub struct RootCycle {
    pub cycle_id: u32,
    pub world_seed: u64,
    pub route_seed: u64,
    pub player_loadout_hash: u64,
    pub events: Vec<CycleEvent>,
    pub end_state: CycleEndState,
}

pub enum CycleEvent {
    HeardNameShear(ErasureEventId),
    InterruptedErasure(ErasureEventId),
}
```

```

FailedErasure(ErasureEventId),
RestoredName(NameId),
PlayerDeath(DeathRecord),
ShadowBorn(ShadowId),
FactionAdvanced(Faction, u16),
RefusalRecorded(RefusalKind),
}

```

8. Combat Data Types

```

pub enum CombatAspect { Ledger, Knife, Root, Vowless }
pub enum CombatPlane { XY, XZ, YZ }
pub enum AlchemicalTier { Nigredo, Albedo, Citrinitas, Rubedo, Aspirational }

pub struct Resonance {
pub hz: i16,
pub inverse: bool,
}

pub struct HarmonicBody {
pub resonance: Resonance,
pub tier: AlchemicalTier,
pub mass_q: i32,
pub amplitude_q: i32,
pub stability_q: i32,
}

```

9. Dissonance Rules

```

pub enum MechanicalDemand {
Aggression, Patience, Mobility, StationaryChannel,
InventoryPrecision, ParryTiming, ResourceHoarding, Sacrifice, Refusal,
}

pub struct QuincunxPressure {
pub demand_a: MechanicalDemand,
pub demand_b: MechanicalDemand,
pub severity: u8,
}

pub struct YodEncounter {
pub apex: Zodiac,
pub base_a: Zodiac,
pub base_b: Zodiac,
pub pressure_a: MechanicalDemand,
pub pressure_b: MechanicalDemand,
}

```

10. Shadow Records

```

pub struct ShadowEcho {
pub cycle_id: u32,
pub source_seed: u64,
pub dominant_aspect: CombatAspect,
pub repeated_inputs_hash: u64,
pub death_position: Vec3i,
pub name_failures: Vec<NameId>,
pub execution_count: u16,
pub refusal_count: u16,
pub route_hash: u64,
}

```

11. Determinism Contract

- No combat-critical floating point math.
- All procedural choices derive from root_seed, cycle_id, zone_id, encounter_id, player_choice_hash, and world_state_hash.
- All erasure schedules are deterministic and replayable.
- Shadow files record inputs, state hashes, route hashes, and memory summaries, not video or massive dumps.
- The current world can be reconstructed from root seed plus event ledger plus verified player inputs.

Rust thesis: IRONROOT is a deterministic 120Hz integer simulation where identity, memory, faction pressure, erasure, and Shadow behavior are all event-sourced records.